

SQL



Database:

Database is an organized collection of logically related data that is stored in an efficient manner so that it can be easily accessed managed and updated.

DBMS – Data Base Management System

The software which is used to manage the database is called Database Management System(DBMS).

It provides us with an interface or a tool, to perform various operations like:

- creating the database

- manipulating the database

- storing and retrieving the data from the database

- deleting data from the database, etc

SQL

SQL or Structured Query Language is simply a computer language that

The database related operations are performed using this Structured Query Language.

Some of the operations that can be performed with the help of SQL are:

Users can create a database.

Users can create some tables in some database.

Users can delete or alter the properties of the table.

Users can provide particular grant access to some particular users using the SQL.

Users can put constraints on the data stored in the database.

Types of SQL statements

- DDL
- DML
- DCL
- TCL
- DQL(Data Retrieval)

DDL - Data Definition Language:

Data Definition Language or DDL is used to define the database structure or schema.

the DDL commands are **CREATE, ALTER, DROP, TRUNCATE, RENAME.**

DML - Data Manipulation Language:

Data Manipulation Language or DML is used to manage the data in the table

the DML commands are INSERT, UPDATE, DELETE

DCL - Data Control Language:

Data Control Language or DCL is used to grant and remove access of schema objects to some users.

DCL commands are GRANT, REVOKE.

TCL - Transaction Control Language:

Transaction Control Language or TCL is used to manage the changes that are made by the DML commands. So, the changes made by the DML commands can be controlled by the DCL statements. Some of the DCL commands are COMMIT, ROLLBACK, SAVEPOINT.

Data Retrieval:

Data Retrieval is used to retrieve or get data from the database. The SELECT statement is used to retrieve data from the database.

Table:

The data in RDBMS is stored in database objects called tables. The table is a collection of related data entries and it consists of columns and rows.

Field:

Every table is broken up, into smaller entities called fields. These are also called as columns or attributes

record :

A record, also called a row of data, is each individual entry that exists in a table is a record

CUSTOMER				
ID	NAME	AGE	ADDRESS	SALARY
1	Ramesh	32	Ahmedabad	2000.00
2	kaushik	23	Mumbai	3000.00

Null:

A NULL value in a table is a value in a field that appears to be blank.
a field with a NULL value is a field with no value.

Constraint:

A constraints is a condition.

SQL constraints are used to specify rules for the data in a table.

Ex:

NOT NULL Constraint: Ensures that a column cannot have NULL value.

Index:

The Index in SQL is a special table used to speed up the searching of the data in the database tables.

Index can be created by using single or group of columns in a table.

Syntax:

```
CREATE INDEX index_name ON table_name ( column1, column2,....);
```

Ex: CREATE INDEX idx_age ON CUSTOMERS (AGE);

MySQL Client:

- MySQL client is a tool or a software which is used to connect to MySQL Server.
- Client programs are used to send commands or queries to the server and allow managing data in the databases stored on the server
- Ex: MySQL Command Line Client, MySQL Workbench

create a user:

Syntax:

```
CREATE USER 'username' IDENTIFIED BY 'password';
```

grant privileges to the new user

Syntax:

```
GRANT ALL PRIVILEGES ON *.* TO 'username';
```


command to create a database:

Syntax:

```
CREATE DATABASE dbname;
```

Ex:

```
create database manhattan;
```

CREATE DATABASE

The CREATE DATABASE statement is used to create a new SQL database.

Syntax

```
CREATE DATABASE databasename;
```

SELECT Database

SELECT Database is used in MySQL to select a particular database to work with.

This query is used when multiple databases are available with MySQL Server.

You can use SQL command **USE** to select a particular database.

Syntax:

```
USE database_name;
```

Start Using Database:

Syntax:

```
use dbname;
```

Ex:

```
use manhattan
```

delete database:

Syntax:

```
drop database dbname;
```

delete user account :

Syntax:

```
DROP USER 'username';
```

Command to clear the Console:

```
System cls
```

Create table:

The CREATE TABLE statement is used to create a new table in a database.

Syntax

```
CREATE TABLE table_name (  
    column1 datatype,  
    column2 datatype,  
    column3 datatype,  
    ....);
```

Ex:

```
CREATE TABLE Persons (  
    PersonID int,  
    LastName varchar(255),  
    FirstName varchar(255),  
    Address varchar(255),  
    City varchar(255)  
);
```

Drop table:

The DROP TABLE statement is used to drop an existing table in a database. **Syntax**

```
DROP TABLE table_name;
```

TRUNCATE TABLE

The TRUNCATE TABLE statement is used to delete the data inside a table, but not the table itself.

Syntax

```
TRUNCATE TABLE table_name;
```

ALTER TABLE Statement

- The ALTER TABLE statement is used to add, delete, or modify columns in an existing table.

ADD Column:

To add a column in a table

Syntax:

```
ALTER TABLE table_name  
ADD column_name datatype;
```

Ex:

```
ALTER TABLE Customers  
ADD Email varchar(255);
```

DROP COLUMN:

To delete a column in a table

Syntax:

```
ALTER TABLE table_name  
DROP COLUMN column_name;
```

ALTER TABLE Statement

Ex

_alter table student modify column age varchar(3);

_alter table student rename to Studnetnew;

alter table student rename to Studnetnew;

Auto increment field:

- Auto-increment allows a unique number to be generated automatically when a new record is inserted into a table.
- MySQL uses the AUTO_INCREMENT keyword to perform an auto-increment feature.

Ex:

```
CREATE TABLE Persons (  
    Personid int NOT NULL AUTO_INCREMENT,  
    LastName varchar(255) NOT NULL);
```


INSERT Statement:

INSERT statement is used to store or add data in MySQL table within the database.

insertion of records in two ways using a single query in MySQL:

- Insert record in a single row
- Insert record in multiple rows

Insert record in a single row

Syntax:

```
INSERT INTO table_name ( col1, col2,...colN )
```

```
VALUES
```

```
( value1, value2,...valueN );
```

Ex:

```
INSERT INTO People (id, name, occupation, age)
```

```
VALUES (101, 'Peter', 'Engineer', 32);
```

Insert record in multiple rows:

Syntax:

```
INSERT INTO table_name VALUES  
( value1, value2,...valueN ),  
( value1, value2,...valueN ),  
.....  
( value1, value2,...valueN );
```

Ex:

```
INSERT INTO People VALUES  
(102, 'Joseph', 'Developer', 30),  
(103, 'Mike', 'Leader', 28),  
(104, 'Stephen', 'Scientist', 45);
```

Update Statement:

The UPDATE statement is used to modify the existing records in a table.

Syntax

```
UPDATE table_name  
SET column1 = value1, column2 = value2, ...  
WHERE condition;
```

Ex:

```
UPDATE Customers  
SET ContactName = 'Sumit choudary', City = 'Pune'  
WHERE CustomerID = 1;
```

DELETE Statement

The DELETE statement is used to delete existing records in a table.

Syntax

```
DELETE FROM table_name WHERE condition;
```

Ex:

ex:

```
DELETE FROM Customers WHERE CustomerName='Manish';
```

MODIFY COLUMN

To change the data type of a column in a table

Syntax:

```
ALTER TABLE table_name
```

```
MODIFY COLUMN column_name datatype;
```

Rename Column Name

```
alter table student rename column id to sid;
```

Rename Table Name

```
alter table studentNew rename Student;
```

PRIMARY Key:

A primary key is a field or column in a table which uniquely identifies each row/record in a database table.

Primary keys must contain unique values.

A primary key column cannot have NULL values.

Primary key may consist of single or multiple fields

Ex:

```
Create table student7(Sno int(3),Sname varchar(10),Marks int(3),PRIMARY KEY(Sno)
);
```

```
CREATE TABLE CUSTOMERS( ID INT ,NAME VARCHAR (20),AGE INT,ADDRESS CHAR
(25), SALARY DECIMAL (18, 2),PRIMARY KEY (ID));
```

After table creation:

if user want to add the primary key column then the command is
`ALTER TABLE CUSTOMER ADD PRIMARY KEY (ID);`

Delete Primary Key:

FOREIGN Key:

A FOREIGN KEY is a field (or collection of fields) in one table that refers to the PRIMARY KEY in another table.

A foreign key is a key used to link two tables together.

Ex:

```
CREATE TABLE Orders (  
    OrderID int ,  
    OrderNumber int ,  
    PersonID int,  
    PRIMARY KEY (OrderID),  
    FOREIGN KEY (PersonID) REFERENCES Persons(PersonID)  
);
```

Note:

A foreign key column in the child table will only accept values which are there in the primary key column of parent table.

UNIQUE Constraint:

The UNIQUE Constraint prevents two records from having identical values in a particular column

In the CUSTOMERS table, for example, you might want to prevent two or more people from having identical SSN.

```
CREATE TABLE CUSTOMERS(  
  ID INT NOT NULL,  
  NAME VARCHAR (20) NOT NULL,  
  SSN INT NOT NULL UNIQUE,  
  ADDRESS CHAR (25) ,  
  SALARY DECIMAL (18, 2),  
  PRIMARY KEY (ID)  
);
```

data types:

SQL data types define the type of value that can be stored in a table column.

SQL data types can be broadly divided into the following categories.

Numeric data types such as: INT, BIGINT, FLOAT, DOUBLE, etc.

Date and Time data types such as: DATE, TIME, DATETIME, etc.

Character and String data types such as: CHAR, VARCHAR, TEXT, etc.

Miscellaneous data types - CLOB, BLOB

Data type	From	to
INT	-2,147,483,648	2,147,483,647
FLOAT	-1.79E+308	1.79E+308
DOUBLE	-3.40E+38	3.40E+38

Data type	Description
DATE	Stores date in the format YYYY-MM-DD
TIME	Stores time in the format HH:MI:SS
VARCHAR	Variable-length storage with a maximum length of 8,000 characters

Logical operators

SQL Logical Operators test for the truth of some given condition. These operators return a **Boolean** value which can be either a TRUE or FALSE.

AND OPERATOR:

AND operator in SQL returns only those records which satisfy both the conditions in the SQL query.

Ex:

```
SELECT * FROM employees WHERE City = "Mumbai" AND Designation = "Project Manager";
```

BETWEEN Operator:

displays the records which fall between the given ranges in the SQL query.

```
SELECT * FROM employees WHERE Salary BETWEEN 50000 AND 90000;
```

OR Operator

The OR operator is used to combine multiple conditions in an SQL statement's WHERE clause.

Ex:

```
SELECT * FROM employees WHERE Designation = "System Engineer" OR  
City = "Mumbai";
```

IN Operator

When we want to check for one or more than one value in a single SQL query, we use IN operator with the WHERE clause in a SELECT query.

Ex:

```
SELECT * FROM employees WHERE City IN ("Mumbai", "Bangalore", "Pune");
```

NOT Operator

NOT operator in SQL shows those records from the table where the criteria is not met.

Ex:

```
SELECT * FROM employees WHERE NOT Designation = "Project Manager"  
;
```

LIKE Operator

LIKE Operator in SQL displays only those data from the table which matches the pattern specified in the query.

Percentage (%) and underscore (_) are the two wildcard operators used with LIKE Operator to perform pattern matching tasks.

```
SELECT * FROM employees WHERE Salary LIKE "5%";
```

SELECT Statement:

The SELECT statement is used to select data from a database. The data returned is stored in a result table, called the result-set.

syntax:

```
SELECT column1, column2, ... FROM table_name;
```

or

```
SELECT * FROM table_name;           ( to display all the columns data)
```

SELECT DISTINCT :

The SELECT DISTINCT statement is used to return only distinct (different) values.(without the duplicates)

Syntax:

```
SELECT DISTINCT columnname FROM tablename;
```

Clauses in Select statement

Where

The where clause is used to filter out data i.e it returns information that satisfies a certain condition.

Ex:

```
Select Student_Name From Student Where Student_Marks >50
```

Group by

This is mostly used with aggregate functions to group the result set according to the value of a column.

example –

```
Select Count (Student_Number), Student_MajorSubject From Student  
Group by Student_MajorSubject
```

Having

This is used along with Group By clause because Where clause could not be used by aggregate functions.

```
Select Count(Student_number), Student_MajorSubject  
From Student  
Group by Student_MajorSubject  
Having Count(Student_Number) > 2
```

Order by

The order by keyword is used to sort the results in ascending or descending order. By default, the order is assumed to be ascending.

ex:

```
Select Student_Name  
From Student  
Where Student_Marks>50  
Order by Student_Marks
```


Columns ALIAS:

Column ALIAS are the user defined column headings given to the required column.

Syntax:

```
SELECT col_name AS alias_name FROM table_name;
```

Ex:

```
SELECT stud_code AS Roll_No FROM Student_info;
```

Note:

Column Alias can be provides for any column.

Column Aliases are temporary.

Arithmetic Operators:

Arithmetic Operators are used for performing mathematical calculations such as Addition, Subtraction, Multiplication, Module, and Division

Operator	Description	Example
+	Addition - Adds values on either side of the operator	$a + b$
-	Subtraction - Subtracts right hand operand from left hand operand	$a - b$
*	Multiplication - Multiplies values on either side of the operator	$a * b$
/	Division - Divides left hand operand by right hand operand	b / a
%	Modulus - Divides left hand operand by right hand operand and returns remainder	$b \% a$

LIMIT Clause

The LIMIT clause is used to specify the number of records to return. The LIMIT clause is useful on large tables with thousands of records.

Syntax:

```
SELECT column_name(s)  
FROM table_name  
WHERE condition  
LIMIT number;
```

UNION

The UNION operator is used to combine the result-set of two or more SELECT statements.

- Every SELECT statement within UNION must have the same number of columns
- The columns must also have similar data types
- The columns in every SELECT statement must also be in the same order

Syntax:

```
SELECT column_name(s) FROM table1
```

```
UNION
```

```
SELECT column_name(s) FROM table2;
```

UNION ALL

The UNION operator selects only distinct values by default. To allow duplicate values, use UNION ALL:

Syntax:

```
SELECT column_name(s) FROM table1  
UNION ALL  
SELECT column_name(s) FROM table2;
```

SQL Aggregate Functions

SQL aggregation function is used to perform the calculations on multiple rows of a single column of a table. It returns a single value.

It is also used to summarize the data.

Types of SQL Aggregation Function:

- COUNT
- SUM
- AVG
- MAX
- MIN

Sample Table:

PRODUCT	COMPANY	QTY	RATE	COST
Item1	Com1	2	10	20
Item2	Com2	3	25	75
Item3	Com1	2	30	60
Item4	Com3	5	10	50
Item5	Com2	2	20	40
Item6	Cpm1	3	25	75
Item7	Com1	5	30	150
Item8	Com1	3	10	30

COUNT FUNCTION

COUNT function is used to Count the number of rows in a database table. It can work on both numeric and non-numeric data types. COUNT function uses the COUNT(*) that returns the count of all the rows in a specified table. COUNT(*) considers duplicate and Null.

Syntax

COUNT(*)

or

COUNT([ALL | DISTINCT] expression)

Example

```
SELECT COUNT(*)  
FROM PRODUCT_MAST;
```

SUM Function

Sum function is used to calculate the sum of all selected columns. It works on numeric fields only.

Syntax

SUM()

or

SUM([ALL|DISTINCT] expression)

Example

```
SELECT SUM(COST)
FROM PRODUCT_MAST;
```


AVG function

The AVG function is used to calculate the average value of the numeric type. AVG function returns the average of all non-Null values.

Syntax

AVG()

or

AVG([ALL|DISTINCT] expression)

Example

```
SELECT AVG(COST)
FROM PRODUCT_MAST;
```

MAX Function

MAX function is used to find the maximum value of a certain column. This function determines the largest value of all selected values of a column.

Syntax

MAX()

or

MAX([ALL|DISTINCT] expression)

Example:

```
SELECT MAX(RATE)
FROM PRODUCT_MAST;
```

MIN Function

MIN function is used to find the minimum value of a certain column. This function determines the smallest value of all selected values of a column.

Syntax

MIN()

or

MIN([ALL | DISTINCT] expression)

Example:

```
SELECT MIN(RATE)
FROM PRODUCT_MAST;
```

The SQL GROUP BY Statement

The GROUP BY statement groups rows that have the same values into summary rows, like "find the number of customers in each country".

The GROUP BY statement is often used with aggregate functions (COUNT(), MAX(), MIN(), SUM(), AVG()) to group the result-set by one or more columns.

Syntax

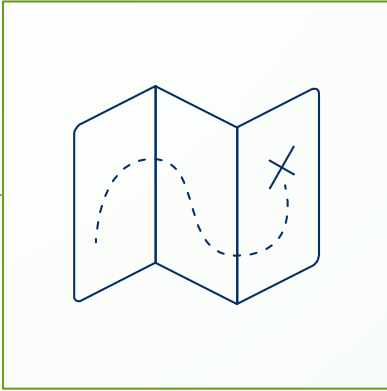
```
SELECT column_name(s)
FROM table_name
WHERE condition
GROUP BY column_name(s)
ORDER BY column_name(s);
```

The SQL HAVING Clause

The HAVING clause was added to SQL because the WHERE keyword cannot be used with aggregate functions.

Syntax

```
SELECT column_name(s)
FROM table_name
WHERE condition
GROUP BY column_name(s)
HAVING condition
ORDER BY column_name(s);
```



THANKS!

Any questions?

You can find me at
sayeedunnisa.js@gmail.com